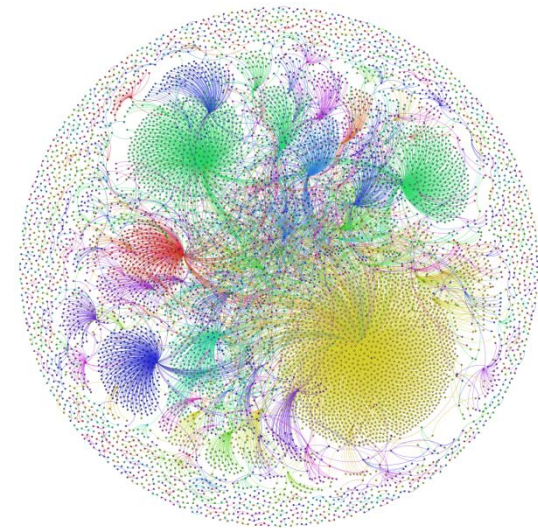


# Structuri de Date

Anul universitar 2019-2020

Prof. Adina Magda Florea



# Curs Nr. 10

---

## **Grafuri - continuare**

- ❑ Grafuri ponderate
  - Algoritmul lui Dijkstra
  - Algoritmul lui Prim
- ❑ Grafuri orientate
  - Închidere tranzitivă
  - Sortare topologica

# 1. Grafuri ponderate

---

- **Ponderi** sau **costuri** asociate arcelor
- Grafuri ponderate neorientate
- Grafuri ponderate orientate
- Găsirea modalității de a conecta toate nodurile cu un cost minim
- Găsirea căii de cost minim între 2 noduri
- Găsirea căii cu număr minim de arce între 2 noduri

# Grafuri ponderate

---

```
typedef struct node
{
    int v; /*indice nod destinatie al unui arc */
    TCost c; /* costul asociat arcului */
    struct node *next;
} TNode, *ATNode;
```

```
typedef struct
/* aloc dinamic vectorul de pointeri */
{
    int nn; /* numar noduri in graf */
    ATNode *adl;
} TGraphL1;
```

# Grafuri ponderate

---

```
typedef struct /* matrice alocata dinamic */  
{  
    int      nn;    /* numar noduri */  
    TCost    **Ma;  
} TGraphM1;
```

```
typedef struct /* vector alocat dinamic */  
{  
    int      nn;    /* numar noduri */  
    TCost    *Ma;  
} TGraphM2;
```

# Algoritmul lui Dijkstra

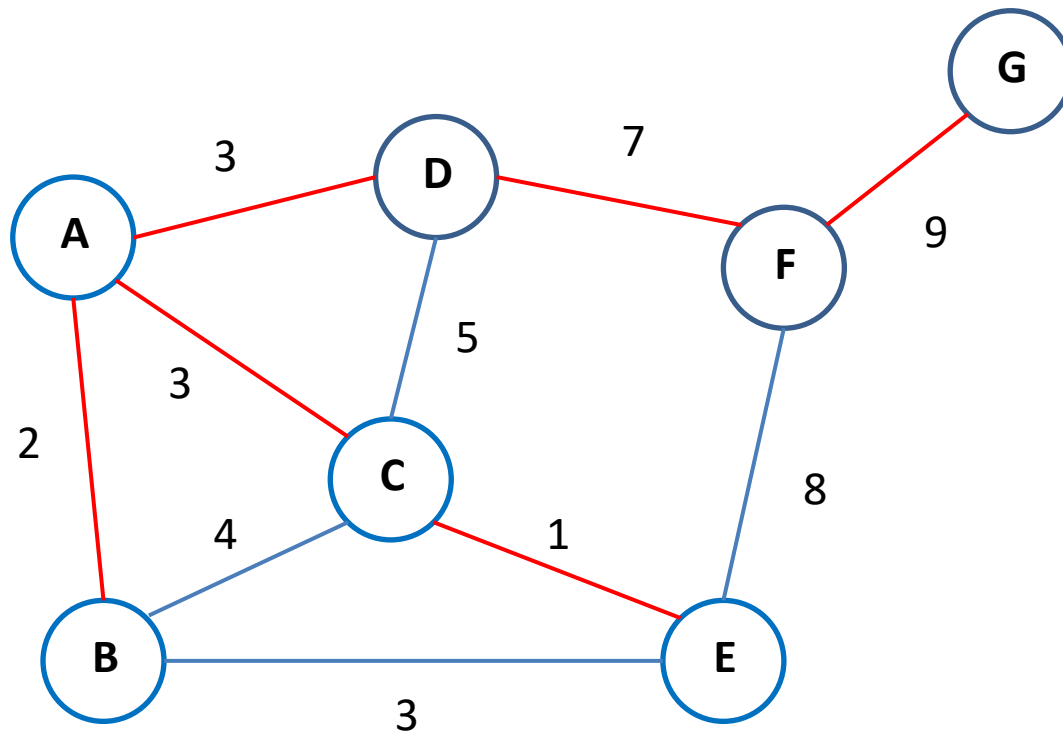
---

- Afla toate **drumurile de cost minim de la un nod (sursa) la celelalte noduri din graf**
  - Graful poate fi orientat sau neorientat
  - Costurile trebuie sa fie nenegative
  - Graful sa fie conex sau tare conex
- 
- **Principiul algoritmului**: Orice subcale a unei cai de cost minim este o cale de cost minim

# Algoritmul lui Dijkstra

---

- Dacă pentru un nod **u** avem un cost de la **u** la **sursa** și am descoperit un nod **v** a.i.  
 $\text{cost\_arc}(u,v) + \text{cost\_drum}(v, \text{sursa}) < \text{cost\_drum}(u, \text{sursa})$   
atunci preferăm calea prin **v**
- Folosește două structuri de date suplimentare
  - Vectorul **val[v]** – distanța între nodul sursă și fiecare nod **v** din graf
  - Vectorul **dad[v]** – părintele fiecărui nod **v** din graf corespunzător unei anumite distanțe



Dijkstra

```

Noduri adiacente nodului A: D cost= 3 C cost= 3 B cost= 2
Noduri adiacente nodului B: E cost= 3 C cost= 4 A cost= 2
Noduri adiacente nodului C: E cost= 1 B cost= 4 D cost= 5 A cost= 3
Noduri adiacente nodului D: F cost= 7 C cost= 5 A cost= 3
Noduri adiacente nodului E: F cost= 8 C cost= 1 B cost= 3
Noduri adiacente nodului F: G cost= 9 E cost= 8 D cost= 7
Noduri adiacente nodului G: F cost= 9
  
```

Cele mai scurte drumuri

```

nod A cost 0 lista tata
nod B cost 2 lista tata A
nod C cost 3 lista tata A
nod D cost 3 lista tata A
nod E cost 4 lista tata C A
nod F cost 10 lista tata D A
nod G cost 19 lista tata F D A
Process returned 7 (0x7) execution time : 37.956 s
  
```





# Algoritmul lui Dijkstra

---

- Folosește o **coada de priorități** cu HeapMin
- Se adaugă o operație  
**pqupdate(q, nod, prioritate)**  
(corespunde partial operației de  
ChangePri(q,i,noua\_p) din Curs 8)
- Modifica această  
**pqupdate(q, nod, prioritate)** astfel:
  - dacă **q** nu există în coadă îl introduc cu prioritate
  - altfel actualizez prioritatea lui **nod** la **prioritate** (ChangePri)

**Algoritm lui Dijkstra** pt a afla toate drumurile de cost minim de la un nod **sursa** la toate celelalte noduri din graf

## **Dijk(sursa)**

Inițializează coada de prioritati

**pentru** fiecare nod **executa**

    val[nod] = unseen (constanta foarte mare)

    dad[nod] = undef (constanta speciala)

val[sursa] = 0

dad[sursa] = 0

enqueue(q, sursa)

**/\* Dijk(sursa) – continuare \*/**

**cat timp** coada nu este vida **repeta**

u = extrage din coada nodul cu valoare minima

**pentru** fiecare vecin v a lui u **executa**

**daca**  $\text{val}[v] > \text{val}[u] + \text{cost\_arc}(u,v)$

**atunci**

$\text{val}[v] = \text{val}[u] + \text{cost\_arc}(u,v)$

$\text{dad}[v] = u$

$\text{pqupdate}(q, v, \text{val}[v])$

**sfarsit**

# Arbore de acoperire

---

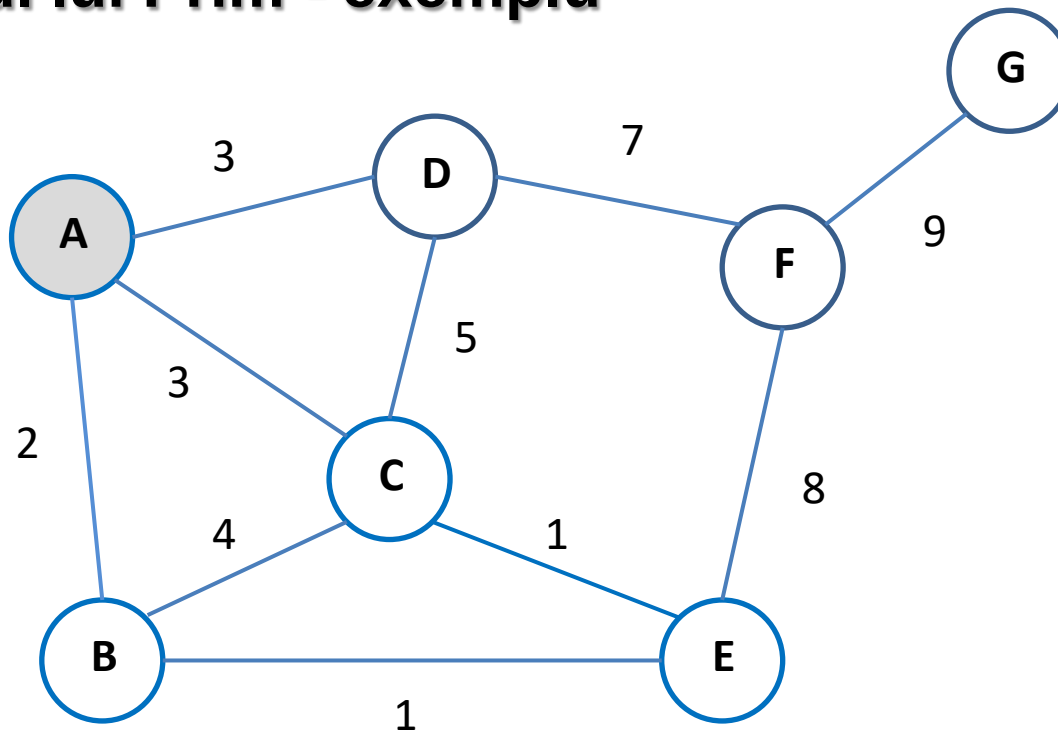
- Un **arbore de acoperire** (spanning tree)  $G' = (V, E')$  al unui graf  $G = (V, E)$  este un subgraf de acoperire ( $E' \subseteq E$ ) care este un arbore liber
- Un **arbore de acoperire de cost minim** (minimum spanning tree) într-un graf ponderat  $G = (V, E)$  este un arbore de acoperire  $A = (V, E')$  a.i. suma costurilor arcelor din  $A$  este mai mică decât sau egală cu suma costurilor arcelor din orice alt arbore de acoperire  $A' = (V, E'')$

# Algoritmul lui Prim

---

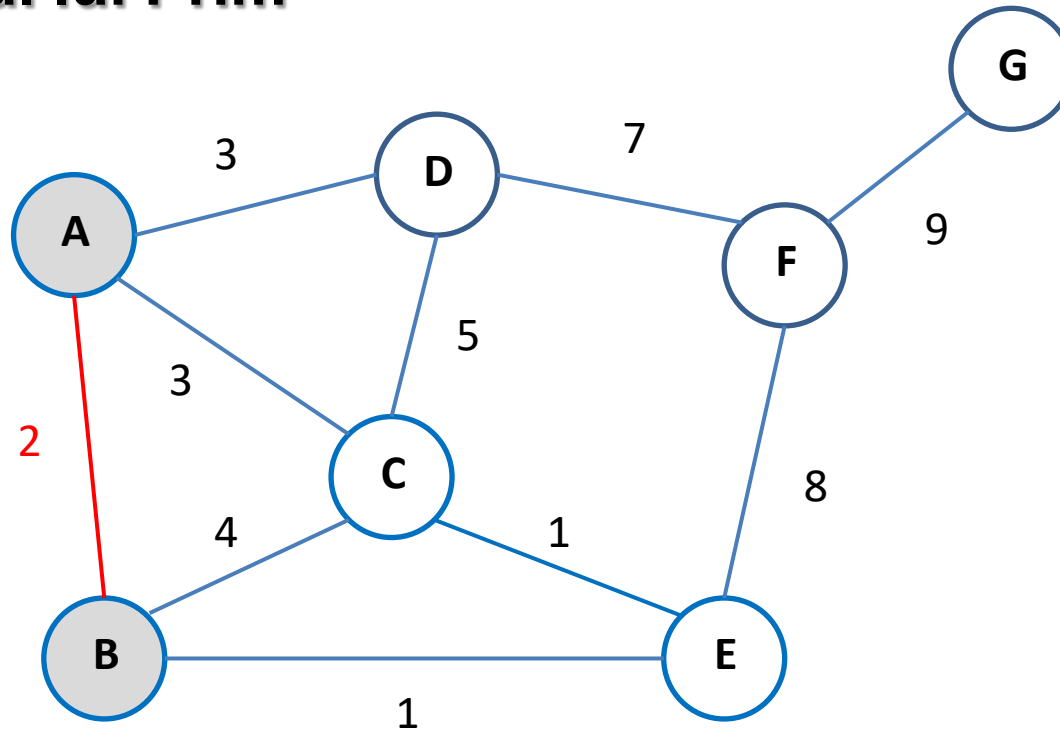
- Determină arborele de acoperire de cost minim
- Incepe cu un arbore vid și încearcă să adauge pe rând câte un arc
- Selectează arbitrar un nod pe post de rădăcină
- Cât timp arborele nu conține toate nodurile din graf, se alege un arc de cost minim legat la arborele parțial construit și se adaugă dacă nu formează cicluri
- Se mențin 2 mulțimi de noduri: cele introduse în arbore (S) și cele neintroduse încă (V-S)

# Algoritmul lui Prim - exemplu



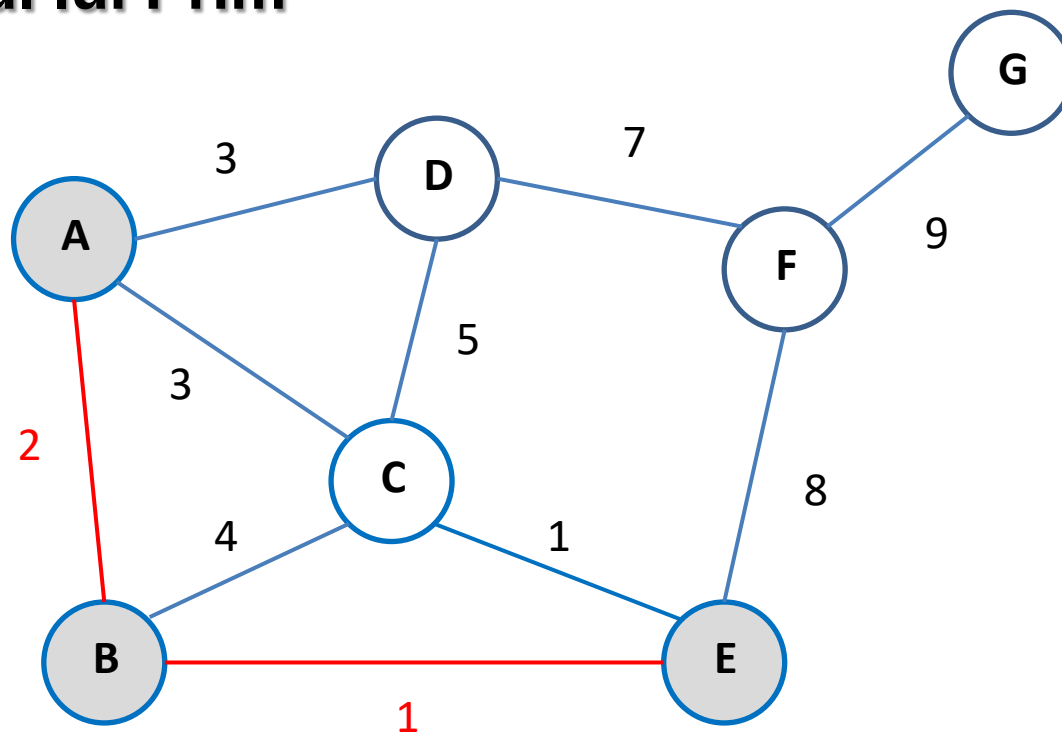
$S = \{A\}$   $V - S = \{B, C, D, E, F, G\}$

# Algoritmul lui Prim



$S = \{A, B\}$   $V - S = \{C, D, E, F, G\}$

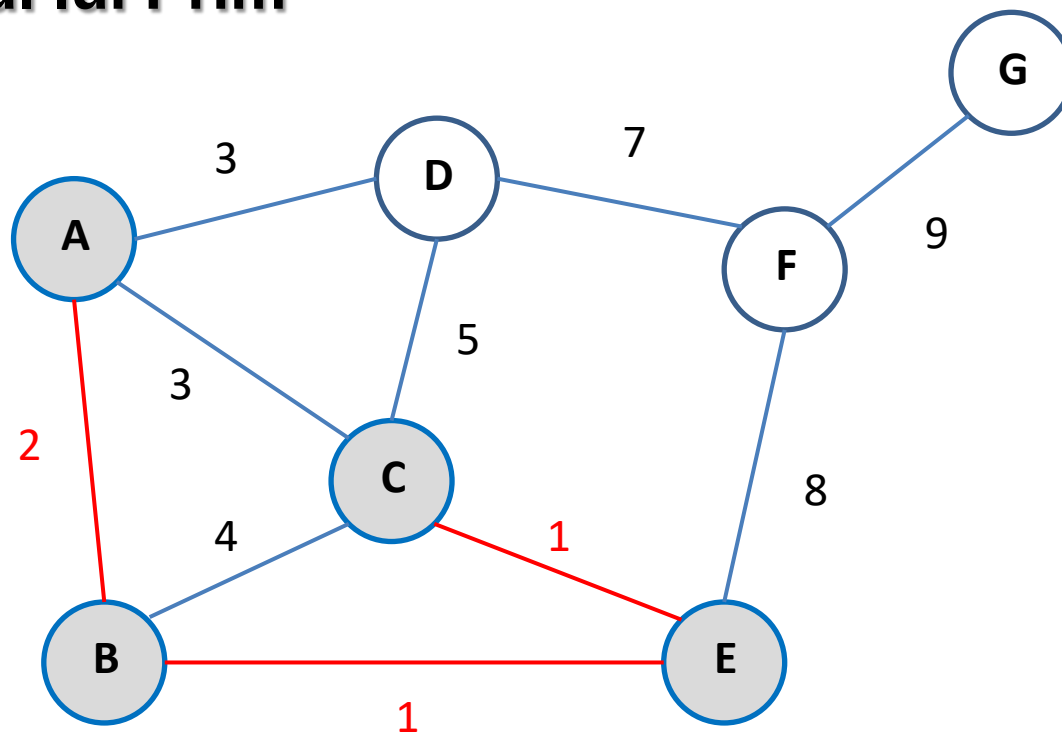
# Algoritmul lui Prim



$S = \{A, B, E\}$   $V - S = \{C, D, F, G\}$

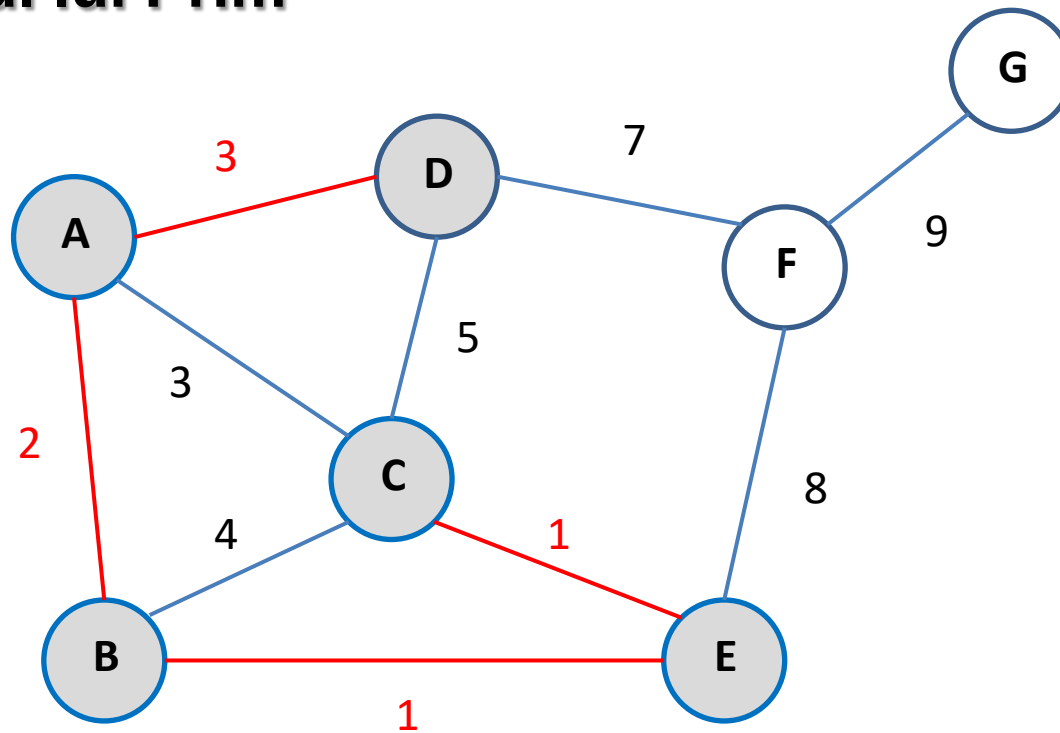


# Algoritmul lui Prim



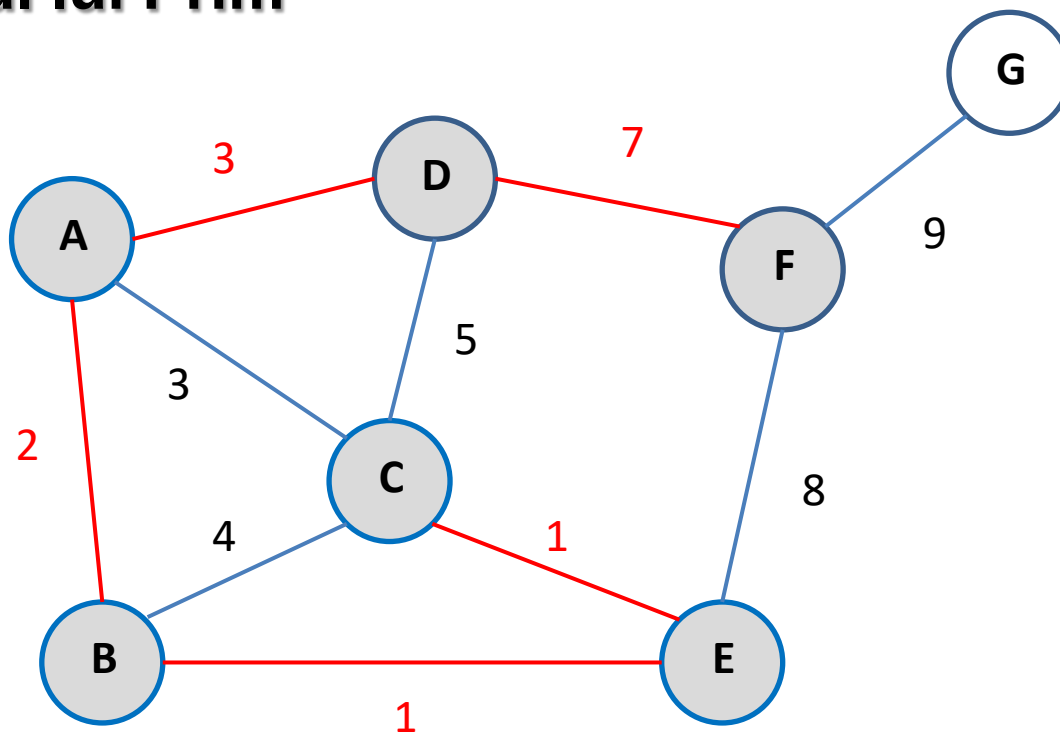
$S = \{A, B, E, C\}$   $V - S = \{D, F, G\}$

# Algoritmul lui Prim



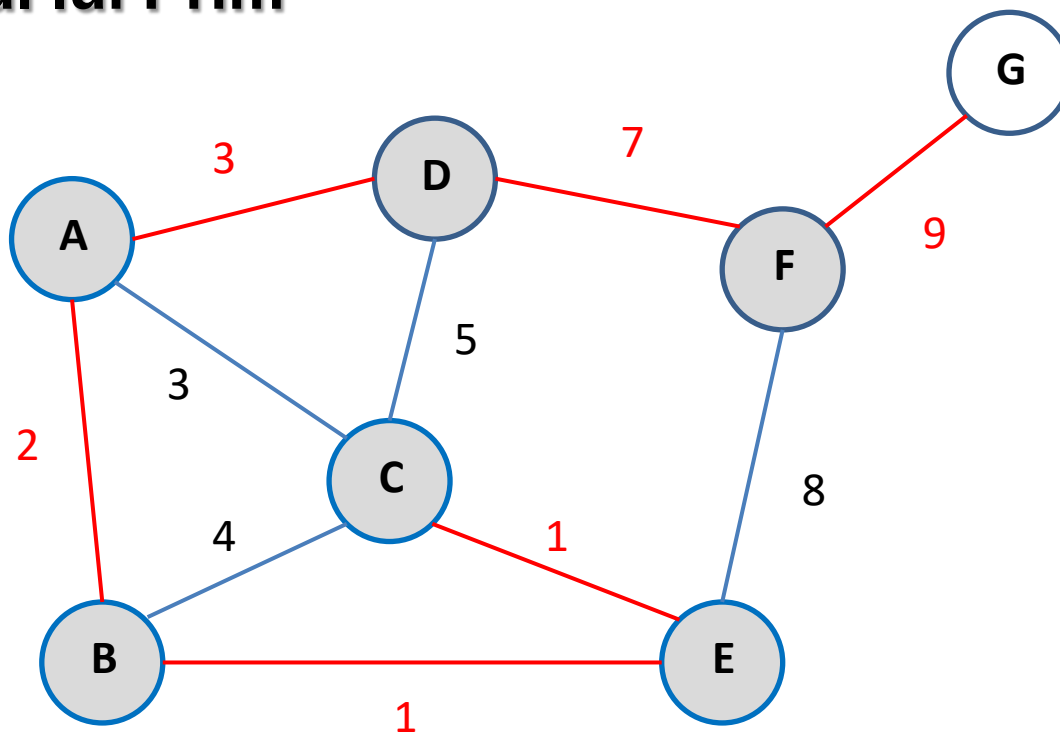
$S = \{A, B, E, D\}$   $V - S = \{F, G\}$

# Algoritmul lui Prim



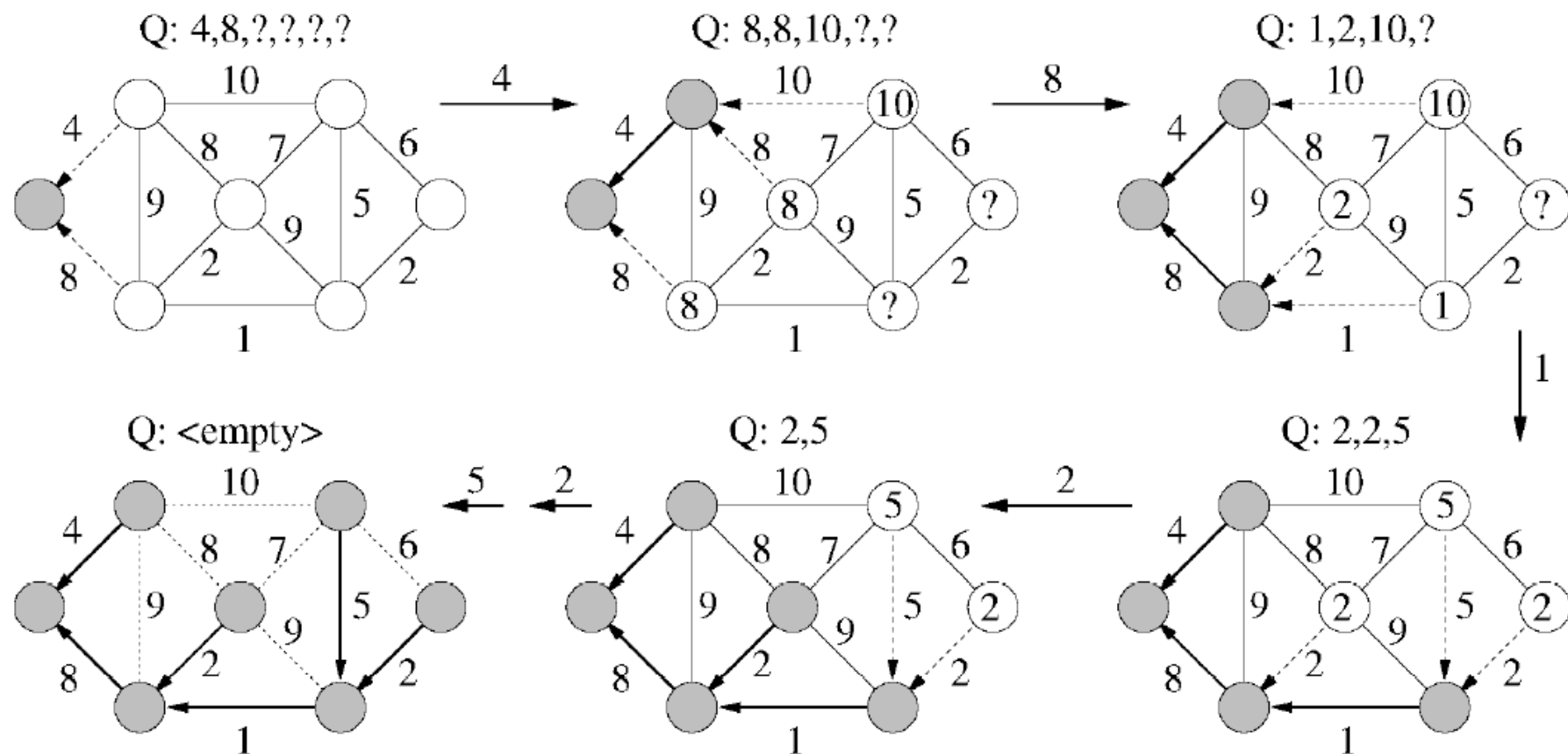
$S = \{A, B, E, D, F\}$   $V - S = \{G\}$

# Algoritmul lui Prim



$S = \{A, B, E, D, F, G\}$   $V - S = \{C\}$

# Algoritmul lui Prim – alt exemplu



# Algoritmul lui Prim

---

- Folosește trei structuri de date suplimentare
  - Vectorul **val[v]** – cea mai mica distanta intre un nod  $v \in V$  si arbore
  - Vectorul **dad[v]** – parintele unui nod  $v \in V$  in arbore
  - Vectorul **color[v]** – este **black** pentru noduri din multimea de noduri deja in arbore **S**, **white** in caz contrar ( $Q=V-S$ )

## Algoritm lui Prim pt a afla arborele de acoperire de cost minim

Initializez S cu vida si Q cu nodurile din graf (color[nod]=white)

**pentru** fiecare nod din graf **executa**

    val[nod]=unseen (val foarte mare)

    dad[nod]=undef (nu exista conexiune)

**cat timp** Q nu este vida **repeta**

    extrage din Q nodul u cu valoare minima val[u]

**daca** color[u] = white **atunci** color[u] = black

**pentru** fiecare vecin v a lui u **executa**

**daca** color[v] = white **si** cost\_arc(u,v) < val[v]

**atunci**

            val[v] = cost\_arc(u,v)

            dad[v] = u

**sfarsit**

**Algoritm lui Prim** pt a afla arborele de acoperire de cost minim (pornind de la nodul sursa)

Foloseste o **coada de priorități** cu HeapMin

**Prim(sursa)**

Inițializează coada de prioritati (noduri cu val[])

**pentru** fiecare nod **executa**

val[nod]=unseen (val foarte mare)

dad[nod]=undef (nu exista conexiune)

color[nod]=white

val[sursa] = 0      dad[sursa] = 0

enqueue(q,sursa)



## Prim(sursa) – cont

**cat timp** coada nu este vida **repeta**

u = extrage din coada nodul cu valoare minima

**pentru** fiecare vecin v a lui u **executa**

**daca** color[v] = white **si** cost\_arc(u,v) < val[v]

**atunci**

val[v] = cost\_arc(u,v)

dad[v] = u

pqupdate(q, v, val[v])

color[u]=black

**sfarsit**

**$O( (V+E) \log V )$**  daca se utilizeaza heap

# Algoritmul lui Kruskal

---

Creeaza o padure de arbori  $A$  in care, initial, fiecare nod din graf este un arbore

Creeaza o multime  $S$  cu toate arcele din graf

**cat timp**  $S$  nu este vida si se mai pot adauga in  $A$   
**repetă**

    Elimina arcul de cost minim din  $S$

**daca** arcul conecteaza 2 arbori diferiti din  $A$   
    **atunci** adauga arcul in  $A$

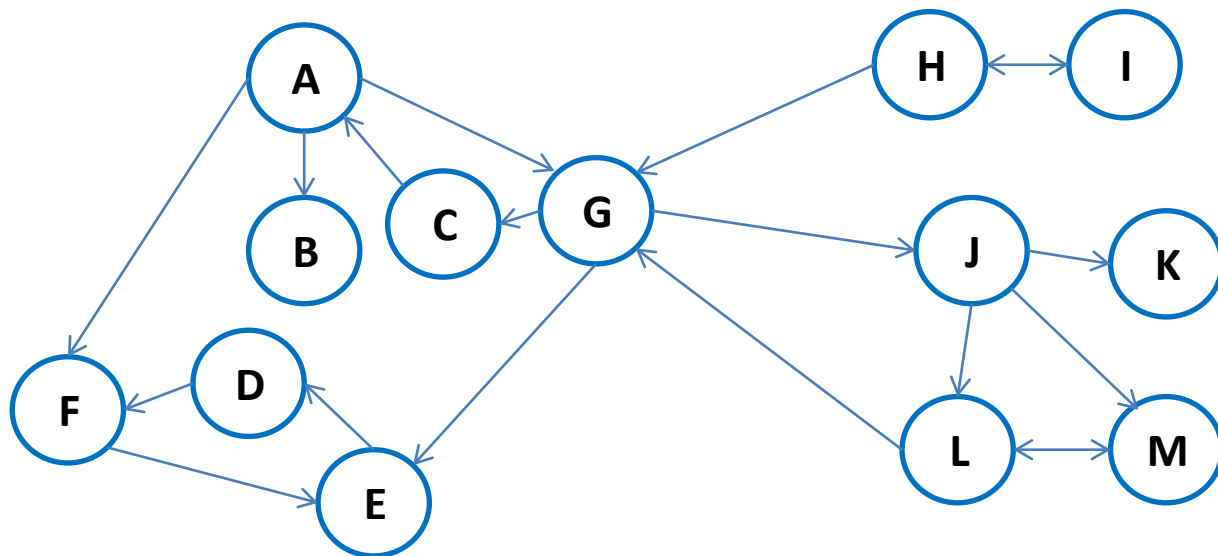
- Padurea contine 1 sau mai multi AAM – unul daca graful este conex

## 2. Grafuri orientate

---

### Grafuri orientate

- In DFS Tree legăturile către noduri întâlnite pot fi de mai multe feluri
- Nu este neapărat necesar ca toate nodurile care sunt accesibile dintr-un anumit nod in graf să fie în arborele DFS care are ca rădăcină acel nod

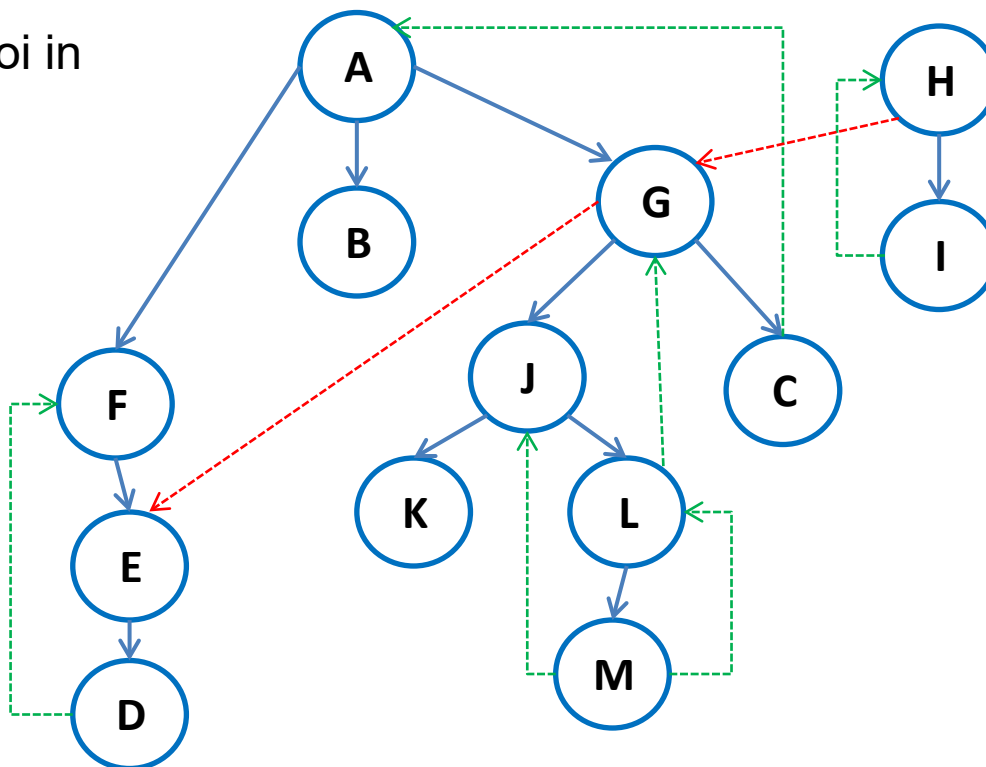


Legături înapoi in  
DFS Tree

Down

Up

Cross



# Grafuri orientate

### 3. Componente tare conexe

---

- Toate nodurile care pot fi accesate din oricare nod din multime
- Apoi vedem daca y este accesibil din x si invers
- Pentru a obține toate nodurile care pot fi accesate dintr-un anumit nod se apelează **visit** de V ori, o dată pentru fiecare nod.

(Se poate obtine si folosind algoritmul lui R.E. Tarjan

[https://en.wikipedia.org/wiki/Tarjan%27s\\_strongly\\_connected\\_components\\_algorithm](https://en.wikipedia.org/wiki/Tarjan%27s_strongly_connected_components_algorithm)

<https://www.geeksforgeeks.org/tarjan-algorithm-find-strongly-connected-components/> )

# Toate nodurile care pot fi accesate dintr-un anumit nod

---

```
void visit(int k)
{
    ATNode t;
    val[k]=++ind;
    for(t = g.adl[k]; t != NULL; t = t->next)
        if(val[t->v]==0) visit(t->v);
}
```

```
void print_toate_accesibile()
{
    int k,i;
    for(k=1; k<=V; k++)
    {
        id = 0;
        for(i=1; i<=V; i++) val[i]=0;
        visit(k);
        printf("\n");
    }
}
```



# Închiderea tranzitivă

---

- Ne interesează pentru anumite probleme **închiderea tranzitivă a unui graf orientat**

- Relație tranzitivă

$$R: A \times A$$

$\forall x, y, z$  dacă  $x R y$  și  $y R z$  atunci  $x R z$

- Închidere tranzitivă a relației  $R$

$$R^k = R \bullet R \dots \bullet R \text{ de } k \text{ ori, } R^{k+1} = R^k$$

- Relație tranzitivă **arc  $x \rightarrow y$**
- Închidere tranzitivă a relației  $R$   **$y$  este accesibil din  $x$**

# Închiderea tranzitivă

---

- Se poate realiza prin DFS in:
  - **$O(V(E+V))$**  pentru grafuri rare
  - **$O(V^3)$**  pentru grafuri dense
- Alta varianta pentru grafuri dense  
**Algoritmul lui Warsall**



# Algoritmul lui Warshall

---

- Pentru fiecare **pereche de noduri (x,y)** a.î. **y este accesibil din x**, adăugăm (dacă nu există) un **arc  $x \rightarrow y$**
- **Algoritmul lui Warshall**
- Reprezentarea grafurilor prin **matrice de adiacență**
- Dacă exista un **k** a.i.  **$x \rightarrow k$  și  $k \rightarrow y$**  atunci  **$x \rightarrow y$**
- Găsește închiderea tranzitivă în  **$O(V^3)$**
- Algoritm Warshall – închiderea tranzitivă – matrice de adiacență
- Algoritm Floyd – cele mai scurte cai într-un graf orientat ponderat matrice de adiacență

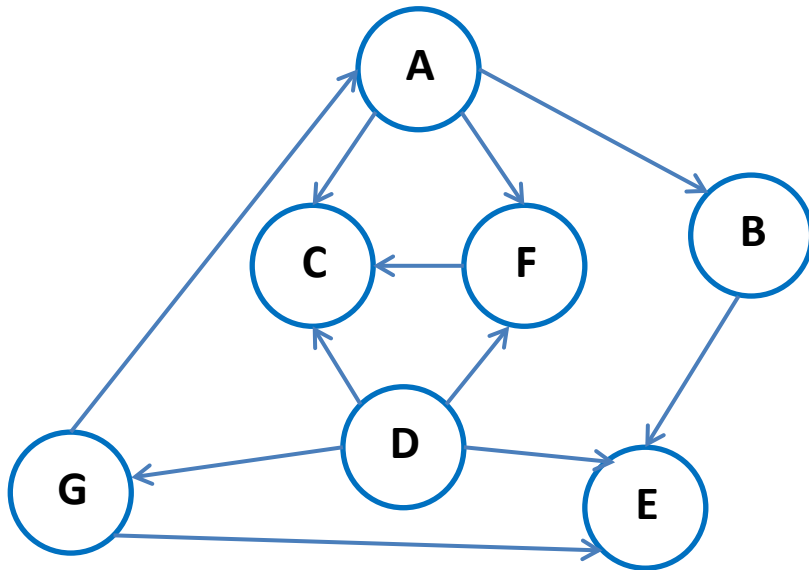
## 4. Sortare topologică

---

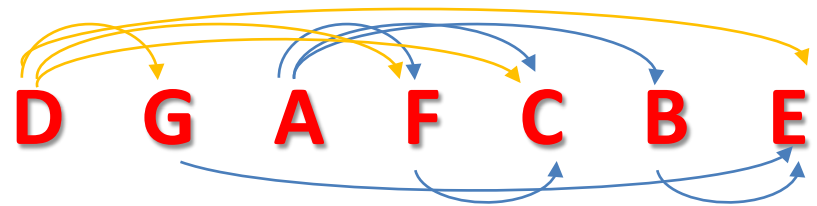
- **DAG** = Directed Acyclic Graph
- **Graf aciclic orientat**
- Pădurea dată de DFS pentru un DAG nu are legături **up**
- O operație fundamentală a DAG este parcurgerea grafului a.î. un nod **x** nu este parcurs înaintea unui nod **y** care punctează la **x**.
- Interpretare: x trebuie executat înaintea lui y
- **Sortare topologică**
- Interpretare: x depinde de y
- **Sortare topologică inversă**

$x \rightarrow y$

# Sortare topologică



**E B C F A G D**



```
Introduceti numar noduri si arce
7 11
A F
A C
A B
B E
D G
D E
D F
D C
F C
G E
G A
a

Reversed topological order
E B C F A G D
Topological order
D G A F C B E
Process returned 7 (0x7)    execution ti
```

# Sortare topologică

---

```
void visit(int k)
{
    ATNode t;
    val[k] = ++ind;
    for(t = g.adl[k]; t != NULL; t = t->next)
        if(val[t->v]== 0) visit(t->v);
    printf(" %c ", k+'A'-1); push(s,k);
}
```

```
void dfs()
{
    int i;
    for(i=1; i<=V; i++) val[i] = 0;
    for(i=1; i<=V; i++)
        if(val[i]==0) visit(i);
}
```

# Sortare topologică

---

```
printf("\nReversed topological order \n");  
    dfs();  
printf("\nTopological order \n");  
    for(i=1;i<=V;i++)  
        printf(" %c ", pop(s)+'A'-1);
```